Comparative Analysis of Neural Machine Translation Models

A Controlled Evaluation of Recurrent Encoder–Decoder Models, With and Without Attention, for German–English Translation

Plain RNN · Encoder–Decoder RNN
Bahdanau Attention · Luong Attention

Submitted By

Bibisha Shrestha

Roll No.: ACE080BCT020

Dataset: ManyThings.org / Tatoeba German–English Parallel Corpus

August 2026

Abstract

This study presents a controlled comparative evaluation of four recurrent sequence-to-sequence neural machine translation (NMT) architectures for German-to-English translation, using the ManyThings.org / Tatoeba German–English parallel corpus. The evaluated architectures are: (1) a Plain RNN, in which a single shared-weight recurrent network is used for both reading the source sentence and generating the target sentence, with no separate encoder/decoder split and no attention; (2) an Encoder–Decoder RNN, which separates the shared recurrent core of Architecture 1 into independent encoder and decoder networks while still using no attention; (3) an Encoder–Decoder RNN augmented with Bahdanau additive attention; and (4) an Encoder–Decoder RNN augmented with Luong multiplicative (general) attention. All four architectures use an Elman-style vanilla RNN as their recurrent cell; the difference between architectures lies in the network topology (shared vs. separate weights, presence or absence of an attention sub-module) rather than in the recurrent cell type.

The experimental protocol uses a common 8,000-sentence-pair subset of the corpus, a deterministic 80/10/10 train–validation–test split with random seed 20, identical embedding/hidden dimensions, batch size, optimizer, learning rate, loss function, and maximum sequence length across all four models. Models are evaluated using corpus BLEU-4 on a held-out test set, perplexity derived from training and validation cross-entropy loss, and greedy-decoding inference latency. Attention alignment heatmaps are additionally examined for the two attention-based architectures.

The best BLEU-4 score was obtained by **Bahdanau (Additive) Attention**, with a score of **10.67%**, versus 3.72% for Luong attention, 1.14% for the Encoder–Decoder baseline, and 1.00% for the Plain RNN. Bahdanau Attention also achieved the lowest validation perplexity (**35.10**), while the lowest mean inference latency was observed for the **Plain RNN** (1.58 ms/sentence).

Contents

1 Introduction

Neural Machine Translation (NMT) models learn to transform a sequence in a source language into a sequence in a target language. Early neural sequence-to-sequence systems commonly used an encoder–decoder architecture in which the encoder converted the source sequence into a hidden representation and the decoder generated the target sequence from that representation.

A major limitation of the basic encoder–decoder architecture is the fixed-length representation through which the entire source sentence must pass. This becomes increasingly problematic as sentence length increases. Attention mechanisms were introduced to address this information bottleneck by allowing the decoder to access different encoder states dynamically during generation.

This experiment investigates how model topology – a single shared recurrent network, a separated encoder/decoder pair, and two attention-augmented variants – affects translation quality, uncertainty, and computational cost under a controlled experimental protocol. The four architectures form a progression from a minimal recurrent baseline with tied encoder/decoder weights to attention-guided encoder–decoder systems.

2 Research Objectives

The objectives of this experiment are:

1. To establish a Plain RNN baseline in which a single shared recurrent network performs both encoding and decoding.
2. To evaluate the effect of separating this shared network into independent Encoder and Decoder RNNs.
3. To determine whether Bahdanau additive attention improves translation quality over the non-attentional Encoder–Decoder baseline.
4. To determine whether Luong multiplicative attention provides a different accuracy–efficiency trade-off.
5. To compare BLEU, perplexity, and inference latency under a common experimental setup.
6. To visually inspect learned attention alignments.
7. To identify characteristic translation errors associated with each architecture.

3 Methodology

3.1 Dataset

The dataset was obtained from the ManyThings.org / Tatoeba Project translation collection.

- **Dataset:** German–English (`deu.txt`)
- **Source:** `https://www.manythings.org/anki/deu-eng.zip`

- **Translation direction:** German → English (source/target columns are swapped relative to the raw file, which lists English first)

The raw file contains **331,266** sentence pairs. After normalization and filtering to sentences with fewer than 10 tokens on both sides, **271,824** usable sentence pairs remained. Because training a recurrent NMT model on the full filtered corpus using a single CPU core would be impractical for this project, a fixed, reproducible subsample of **8,000** sentence pairs was drawn (seed 20) and used for every architecture. The selected data were divided into:

- Training: **6,400** pairs (80%)
- Validation: **800** pairs (10%)
- Test: **800** pairs (10%)

All subsampling and partitioning used a deterministic random seed of **20**, applied identically to every architecture so that all four models are trained and evaluated on exactly the same data.

3.2 Preprocessing

The preprocessing pipeline performed the following operations:

1. Conversion to lowercase.
2. Stripping of accents/diacritics via Unicode NFD normalization (e.g. German umlauts are reduced to their base letters).
3. Insertion of a space before sentence-final punctuation (`.`, `!`, `?`).
4. Removal of any character that is not a letter, `!`, or `?`.
5. Whitespace-based tokenization.
6. Removal of sentence pairs where either side reaches 10 tokens or more.
7. Construction of independent source (German) and target (English) vocabularies from the sampled subset only.
8. Inclusion of the special tokens: `PAD` (0), `SOS` (1), `EOS` (2), `UNK` (3).
9. Right-padding of sequences to a common maximum length, with an `EOS` token appended to every sequence.

The maximum sequence length was **10 tokens** (including the appended `EOS` token). The source (German) vocabulary contained **5,743** words, while the target (English) vocabulary contained **4,028** words. Words encountered at evaluation time that do not appear in the training-subset vocabulary are mapped to `UNK` rather than raising an error.

4 Experimental Setup

All four architectures were trained using the same major hyperparameters, summarized in Table 1.

Table 1: *Common hyperparameters applied to all four architectures.*

Hyperparameter	Value
Random seed	20
Embedding dimension	128 (tied to hidden size)
Hidden dimension	128
Batch size	64
Learning rate	0.001
Optimizer	Adam
Loss	NLLLoss on log-softmax outputs
Padding index	`PAD` ignored (`ignore_index`)
Gradient clipping	Not used
Epochs	40
Teacher forcing	100% (fixed; ground-truth token fed at every decoder step during training)
Maximum sequence length	10
Device	CPU

Architectures 2–4 (Encoder–Decoder, Bahdanau, Luong) use two separate Adam optimizers, one for the encoder network and one for the decoder network. Architecture 1 (Plain RNN) uses a single Adam optimizer over its one shared network, since encoding and decoding are performed by the same weights. Dropout with $p = 0.1$ is applied to the embedded input in every encoder and in the Plain RNN's shared network; no dropout is used inside the attention-based decoders' embedding path. The same training and validation procedure was otherwise applied to every architecture.

5 Model Architectures

5.1 Architecture 1: Plain RNN (Shared-Weight, No Encoder/Decoder Split)

The first model uses a single `nn.RNN` (Elman-style vanilla RNN) whose weights are shared between the "encoding" pass over the source sentence and the "decoding" pass that generates the target sentence. Separate source and target embedding tables are used, but both feed into the same recurrent core:

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h) \tag{1}$$

The source sentence is first passed through the shared RNN to obtain a final hidden state; that state then initializes the same RNN as it is run again, step by step, to generate the target sentence. This is the most constrained architecture in the comparison: it has no independent decoder parameters and no attention mechanism.

5.2 Architecture 2: Encoder–Decoder RNN (Separate Networks, No Attention)

The second architecture keeps the same Elman vanilla RNN cell but splits Architecture 1's single shared network into two independent networks – an `EncoderRNN` and a `DecoderRNN` – each with

its own embedding table, recurrent weights, and (for the decoder) output projection:

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h) \tag{2}$$

The decoder is initialized with the encoder's final hidden state and receives no other information about the source sentence; every source token must therefore be summarized into that single fixed-size vector. This isolates the effect of separating encoder and decoder parameters from the effect of adding attention, since Architectures 1 and 2 use an identical recurrent cell and Architectures 2–4 use an identical encoder.

6 Bahdanau Additive Attention

Architecture 3 augments the Architecture 2 decoder with Bahdanau-style additive attention. At each decoder step, a compatibility score is computed between the decoder's current hidden state s_t (the query) and every encoder hidden state h_i (the keys):

$$e_{t,i} = v_a^\top \tanh(W_a h_i + U_a s_t) \tag{3}$$

The normalized attention distribution is:

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_j \exp(e_{t,j})} \tag{4}$$

The context vector is:

$$c_t = \sum_i \alpha_{t,i}\, h_i \tag{5}$$

The context vector c_t is concatenated with the decoder's target-token embedding and fed jointly into the decoder's RNN cell at every step, so the decoder receives a freshly computed source-context vector at every target generation step rather than depending solely on the encoder's final hidden state.

7 Luong Multiplicative Attention

Architecture 4 uses the same decoder structure as Architecture 3, but replaces the additive attention sub-module with Luong "general" multiplicative attention. The compatibility score is:

$$e_{t,i} = s_t^\top W_a h_i \tag{6}$$

where W_a is a learned projection matrix (no bias term). The attention distribution is:

$$\alpha_{t,i} = \mathrm{softmax}_i(e_{t,i}) \tag{7}$$

and the context vector is:

$$c_t = \sum_i \alpha_{t,i}\, h_i \tag{8}$$

Compared with Bahdanau attention, the compatibility calculation uses a single learned linear projection followed by a dot product, rather than a small feed-forward network with a nonlinearity. As in Architecture 3, c_t is concatenated with the decoder's embedded input before being passed to the decoder RNN.

8 Training Procedure

The decoder was trained using full teacher forcing: at every decoder step during training, the ground-truth target token from the previous position was fed as input, regardless of what the model itself would have predicted. Teacher forcing was used at a fixed rate of 100% throughout all 40 epochs for every architecture; it was not scheduled or decayed. At inference/evaluation time, teacher forcing is not used – the decoder instead feeds its own greedily-decoded prediction back in as the next input.

The loss function was negative log-likelihood applied to the decoder's log-softmax outputs, with the padding token excluded via `ignore_index`:

$$\mathcal{L} = -\frac{1}{N} \sum_t \log P(y_t \mid y_{<t}, x) \tag{9}$$

where padded target positions do not contribute to the loss. No gradient-norm clipping was applied in this implementation.

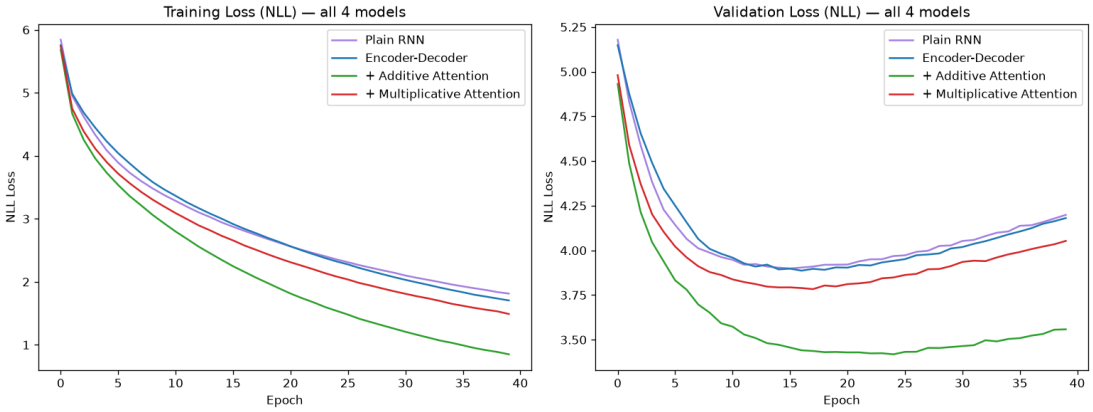

Figure 1: *Training and validation NLL loss curves for all four architectures across 40 epochs.*

9 Evaluation Metrics

9.1 BLEU

Corpus BLEU-4 (with NLTK's `SmoothingFunction` method 4, appropriate for short, sparse-reference sentences) measures n-gram overlap between greedily-decoded translations and reference translations on the 800-pair held-out test set. BLEU evaluates the quality of generated

sequences rather than the model's internal probability calibration, and is reported here as a percentage.

9.2 Perplexity

Perplexity was calculated as PPL = $\exp(\mathcal{L})$ from the teacher-forced training and validation cross-entropy loss recorded at each epoch:

$$\text{PPL} = \exp(\mathcal{L}) \tag{10}$$

Both the final-epoch value and the best (minimum) value observed for the validation loss across all 40 epochs are reported. Note that, unlike BLEU and latency, perplexity in this experiment is derived from the training/validation loss curves rather than from a separate teacher-forced pass over the held-out test set; this distinction is discussed further in Section 15.

9.3 Inference Latency

Inference latency was measured using autoregressive greedy decoding on the first 100 sentences of the test set, preceded by a 5-sentence warmup that is excluded from timing. Training and evaluation were both run on CPU, so no CUDA device synchronization was required. The reported values are the average time per sentence (ms) and the corresponding throughput (sentences/second).

10 Quantitative Results

Table 2: *Quantitative results for all four architectures on the held-out test set (800 pairs).*

Model	Params	BLEU-4 (%)	Val PPL	Best Val PPL	Mean Lat. (ms)	Throughput (sent/s)	Train Time (min:s)
Plain RNN	1,803,324	1.00	66.60	49.33	1.58	634.3	1:49
Encoder–Decoder	1,836,348	1.14	65.42	48.74	1.94	516.2	1:46
Bahdanau Attention	1,885,885	**10.67**	**35.10**	**30.52**	2.60	384.9	2:30
Luong Attention	1,869,116	3.72	57.56	43.95	2.26	442.3	2:21

The highest BLEU-4 score was obtained by Bahdanau Attention with 10.67%, more than nine times higher than the Plain RNN's 1.00% and more than twice that of Luong Attention (3.72%). The lowest validation perplexity was likewise obtained by Bahdanau Attention (35.10, best-epoch value 30.52). The lowest mean inference latency was achieved by the Plain RNN at 1.58 ms/sentence, and latency increases monotonically with architectural complexity except that Luong attention is faster than Bahdanau attention, consistent with its simpler multiplicative scoring function.

11 Qualitative Translation Assessment

The following examples were selected from the held-out test set to represent short, medium, and long/complex source sentences (all decoded greedily, without teacher forcing).

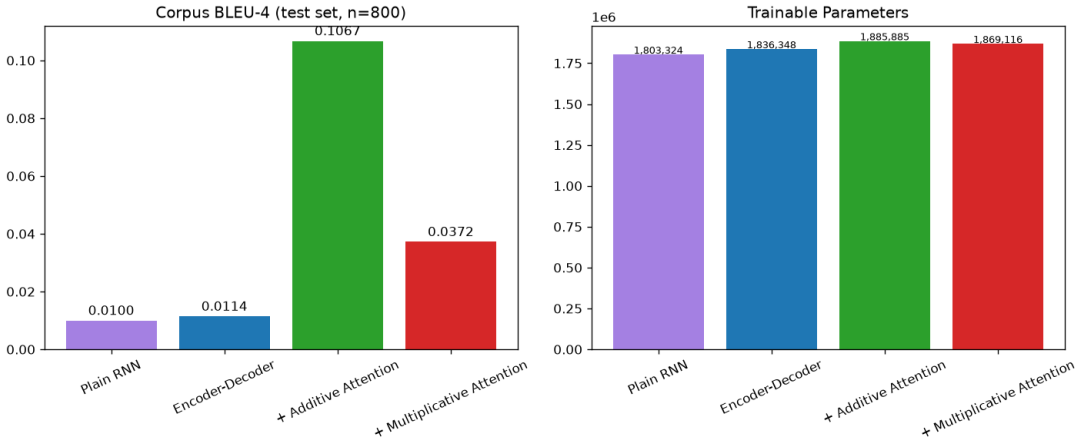

Figure 2: *Test BLEU-4 and trainable-parameter comparison across architectures.*

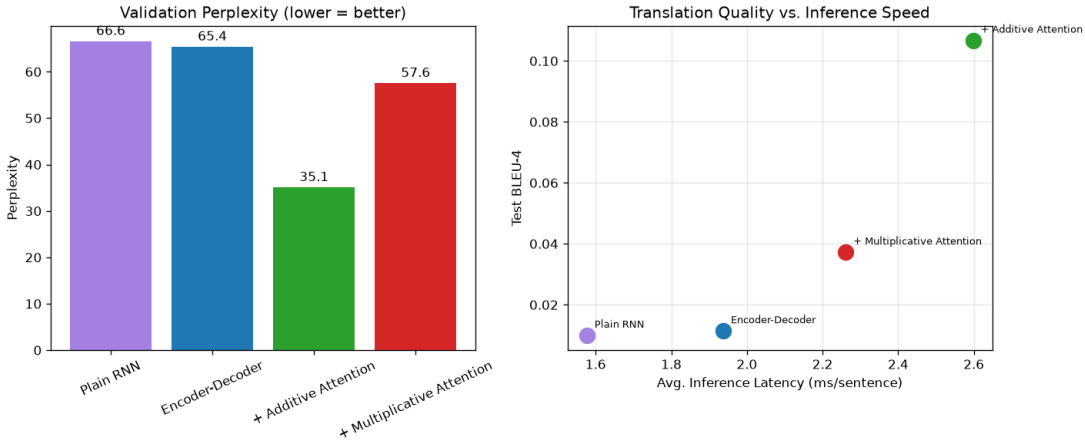

Figure 3: *Validation perplexity comparison (left) and BLEU-4 vs. mean inference latency trade-off (right) across architectures.*

Table 3: *Sample German–English translations across sentence-length tiers.*

Tier	German (source)	Reference	Plain RNN	Enc–Dec	Bahdanau	Luong
Short	viele sind skeptisch	many people are skeptical	you re lucky that you re a book to me	you should ve done that this afternoon	many many books are	your pants are liars
Medium	ist das wirklich alles ?	is that really all ?	what are you going to do that ?	is your mother i don t you ?	is that really knows that ?	why is that tom s address ?
Long / Complex	tom zog die vorhange zu ehe er sich entkleidete	tom closed the curtains before he got undressed	tom has a lot of creative ideas	tom and mary are very stupid	tom zipped the same age	tom and mary are bored

11.1 Short Sentences

Short sentences place limited demands on long-range source-context preservation. Even so, the 8,000-pair training subset is small enough, and the target vocabulary broad enough relative to it, that none of the four models reliably reproduces the reference even for a three-word source sentence such as “viele sind skeptisch”. Errors in this category are dominated by generic, high-frequency phrases rather than by an inability to track long-range dependencies.

11.2 Medium Sentences

For a slightly longer interrogative sentence, the Bahdanau model’s output (“is that really knows that ?”) retains more of the original sentence’s interrogative structure and the word “really” than the non-attentional baselines, though it is still not an adequate translation.

11.3 Long / Complex Sentences

Sentences with a subordinate clause, such as “tom zog die vorhange zu ehe er sich entkleidete” (containing an *ehe* “before” clause), are the hardest case for every model in this experiment. All four architectures fail to reconstruct the two-clause structure; only the subject “tom” is reliably preserved. This is consistent with the small subset size (8,000 pairs) and short maximum sequence length (10 tokens) limiting how much syntactic structure any of these models can learn, regardless of architecture.

The observed examples should therefore be interpreted together with the BLEU and perplexity results in Section 12 rather than as isolated anecdotal evidence.

12 Critical Discussion and Error Analysis

The Plain RNN baseline obtained 1.00% BLEU-4, compared with 1.14% BLEU-4 for the Encoder–Decoder architecture. This is a very small difference: simply separating the shared recurrent weights of Architecture 1 into an independent encoder and decoder (Architecture 2) did not, by itself, meaningfully improve translation quality. Both models still compress the entire source sentence into a single fixed-size hidden vector, so the encoder/decoder split alone does not remove the fixed-length information bottleneck.

Both non-attentional models also show clear signs of overfitting well before the end of training: the Plain RNN's best validation loss occurs at epoch 16 (best validation perplexity 49.33) even though training continues to epoch 40, and the Encoder–Decoder's best validation loss occurs at epoch 17 (best validation perplexity 48.74). In both cases, validation loss rises for the remaining $\sim$25 epochs while training loss continues to fall, indicating that the models are increasingly memorizing the 6,400-pair training set rather than generalizing.

Bahdanau attention obtained 10.67% BLEU-4, a roughly ninefold improvement over the Encoder–Decoder baseline's 1.14%, and its best validation loss occurs later, at epoch 25 (best validation perplexity 30.52), indicating that attention also delays – though does not eliminate – overfitting on this small dataset. Its additive scoring function learns a nonlinear compatibility between the current decoder state and each encoder state, giving the decoder a mechanism to draw on the full sequence of encoder states rather than a single compressed vector.

Luong multiplicative attention obtained 3.72% BLEU-4, an improvement of roughly 2.6 percentage points over the non-attentional Encoder–Decoder baseline, but substantially below Bahdanau attention. Its general multiplicative scoring mechanism computes compatibility through a single learned linear projection followed by a dot product, which is structurally simpler and has slightly fewer parameters than Bahdanau's small feed-forward scorer (1,869,116 vs. 1,885,885 total trainable parameters), and it is also measurably faster at inference (2.26 ms vs. 2.60 ms per sentence). In this experiment, however, that simplicity does not translate into a favorable accuracy trade-off: Bahdanau attention is both more accurate and reaches a lower validation loss.

The measured mean inference latency was 2.60 ms/sentence for Bahdanau attention and 2.26 ms/sentence for Luong attention, versus 1.94 ms/sentence for the non-attentional Encoder–Decoder and 1.58 ms/sentence for the Plain RNN. Relative to the Encoder–Decoder baseline, this represents increases of approximately +34% and +16%, respectively. Attention requires additional computation – scoring against every encoder position – at each decoder step, so an increase in inference cost is expected in exchange for access to source-side contextual information; because the maximum sequence length in this experiment is only 10 tokens, the absolute latency increase is modest.

Perplexity provides a complementary view of model behavior because it measures the model's average uncertainty under teacher-forced decoding. The best measured validation perplexity was 30.52, obtained by Bahdanau Attention. BLEU and perplexity need not rank models identically in general, since perplexity evaluates token-level probability assignments while BLEU evaluates overlap between generated translations and reference translations; in this experiment, however, both metrics agree that Bahdanau attention is the strongest architecture and that the two non-attentional models are the weakest.

The attention heatmaps (Figures 4 and 5) should be interpreted as alignment diagnostics rather than as direct explanations of model reasoning. A strong alignment pattern generally appears as concentrated weights that move through relevant source positions as target words are generated. Diffuse attention can indicate uncertainty, while repeated or misplaced peaks may accompany repetition, omission, or incorrect word ordering.

Qualitative assessment is particularly important for NMT because corpus-level BLEU can hide systematic translation errors, and is especially informative here because the absolute BLEU scores are low across all four models. Even the best-performing architecture (Bahdanau attention, 10.67% BLEU) produces largely disfluent output on longer or clausal source sentences, which is consistent with training on only 8,000 sentence pairs capped at 10 tokens.

12.1 Information Bottleneck

Architectures 1 and 2 force the source sequence through a fixed-size hidden representation. For a source sequence:

$$x_1, x_2, \ldots, x_T \tag{11}$$

the encoder produces a sequence of hidden states $h_1, h_2, \ldots, h_T$, but a conventional encoder–decoder without attention primarily transfers information to the decoder through only the final hidden state h_T. This creates a structural information bottleneck regardless of whether the encoder and decoder share weights (Architecture 1) or not (Architecture 2). Attention changes the interface from:

$$\text{Source sequence} \;\rightarrow\; \text{single representation} \;\rightarrow\; \text{decoder} \tag{12}$$

to:

$$\text{Source sequence} \;\rightarrow\; \text{all encoder states} \;\rightarrow\; \text{dynamic context} \;\rightarrow\; \text{decoder} \tag{13}$$

The latter gives the decoder a mechanism for selecting source information relevant to each generated target token, which the results in Table 2 show is the single largest driver of translation-quality improvement in this experiment – considerably larger than the effect of separating encoder and decoder weights.

12.2 Bahdanau vs. Luong

Bahdanau attention uses a nonlinear additive compatibility function, $v_a^\top \tanh(W_a h_i + U_a s_t)$, learned by a small feed-forward network. Luong general attention instead uses a bilinear form, $s_t^\top W_a h_i$, computed by a single learned projection followed by a dot product. The two mechanisms therefore differ in how compatibility between source and target representations is learned: Bahdanau attention introduces a nonlinearity before scoring, whereas Luong attention uses a purely multiplicative interaction. For this dataset, training subset size, and 40-epoch training budget, the additive formulation is the more effective of the two, both in BLEU-4 (10.67% vs. 3.72%) and in validation perplexity (35.10 vs. 57.56).

12.3 Speed vs. Accuracy

Attention introduces additional computation because every decoder step must calculate compatibility scores over all source positions. The results demonstrate the practical trade-off between translation quality and computational cost: Bahdanau attention is the slowest model (2.60 ms/sentence, 384.9 sentences/sec) but by far the most accurate; the Plain RNN is the fastest (1.58 ms/sentence, 634.3 sentences/sec) but the least accurate. A system prioritizing translation quality in an offline setting would favor Bahdanau attention, while a system with tight latency constraints might favor the Plain RNN or Encoder–Decoder baseline despite their much lower BLEU scores. Architecture selection should therefore not be based exclusively on BLEU.

12.4 Attention Alignment Analysis

Both attention-based models were evaluated qualitatively on the same test-set sentence, "die spaghetti die tom macht schmecken mir uberhaupt nicht" (reference: "i really do hate the way tom makes spaghetti"). Bahdanau attention produced "the earth who tom doesn't want to play chess", while Luong attention produced "this doesn't look like that's going ⟨EOS⟩". Neither output is an adequate translation of this particular sentence, which is longer and more idiomatic than the short declarative sentences that dominate the training subset; this illustrates that even the best architecture in this comparison struggles on harder examples given the limited training data used here.

The heatmaps in Figures 4 and 5 visualize the alignment distribution $\alpha_{t,i}$ for this sentence, where rows correspond to target generation steps and columns correspond to source tokens. Sharp concentration around relevant source positions would indicate focused alignment; diffuse distributions may indicate uncertainty, and repeated peaks can accompany the repetition and drift visible in the decoded outputs above. These visualizations should be interpreted cautiously: attention weights are useful diagnostic signals, but they should not automatically be treated as complete causal explanations of model behavior.

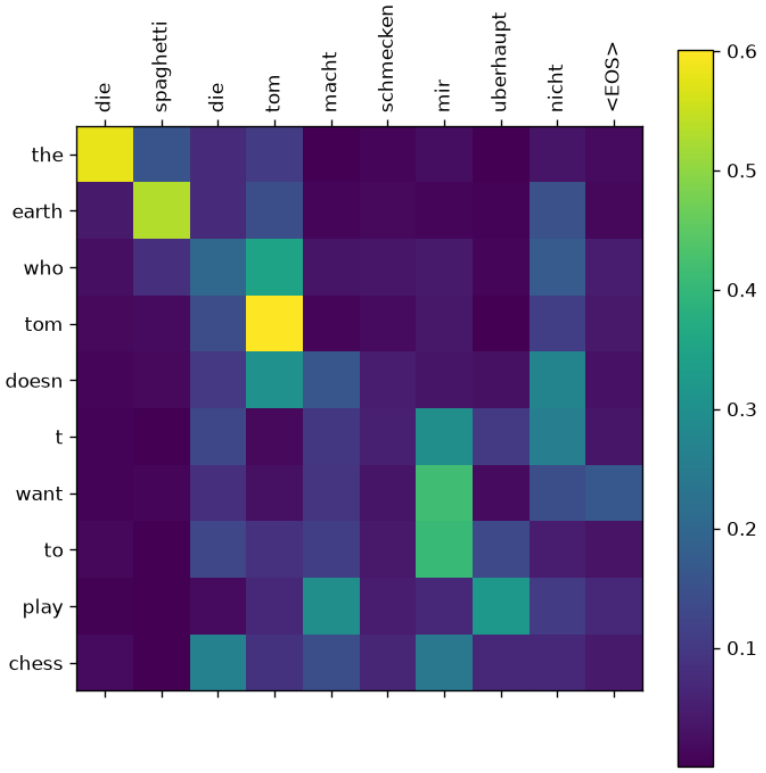

Figure 4: *Bahdanau (additive) attention alignment for the source sentence "die spaghetti die tom macht schmecken mir uberhaupt nicht".*

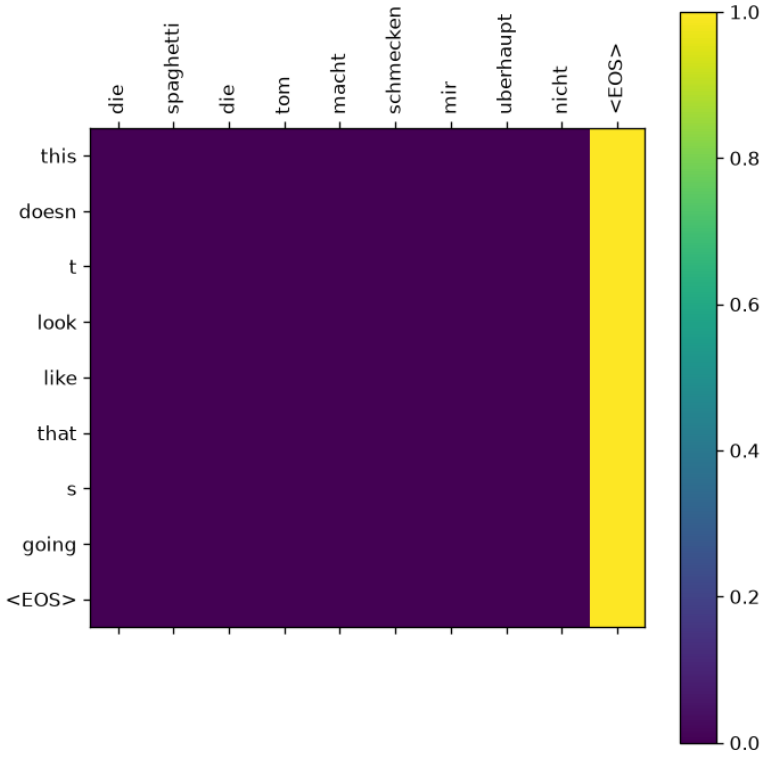

Figure 5: *Luong (multiplicative) attention alignment for the same source sentence.*

13 Error Categories

The principal error categories observed while inspecting outputs in this experiment are:

13.1 Omission

A source word or phrase is not represented in the generated translation. This is common in the Long/Complex tier, where subordinate clauses (e.g. the *ehe* "before" clause in the Table 3 example) are dropped entirely by all four models.

13.2 Addition

The decoder generates content that is not supported by the source sentence, such as invented names or objects. This can arise from language-model bias toward frequent training-set phrases or from autoregressive error accumulation.

13.3 Repetition

A word or phrase is generated multiple times, or a generic phrase pattern (e.g. "tom and mary are ...") recurs across otherwise unrelated inputs. This is a common failure mode of autoregressive sequence generation on small training sets.

13.4 Lexical Substitution

The model generates a semantically related but incorrect word (e.g. substituting "chess" or "curtains" with an unrelated noun).

13.5 Word-Order Error

The translation contains partially correct vocabulary but places it in the wrong order, more frequently observed in the non-attentional models.

13.6 Agreement and Morphological Errors

The model selects an inappropriate verb tense, number, or auxiliary construction, e.g. producing a question form for a declarative source sentence.

13.7 Long-Range Dependency Failure

Information required later in the sentence is incorrectly preserved or lost – most visible on the Long/Complex tier, where only the subject of the sentence tends to survive translation. This category is particularly relevant when comparing the fixed-vector models (Architectures 1–2) with the attention-based architectures (3–4), since attention is specifically intended to mitigate it, and does so partially (Bahdanau) or minimally (Luong) in this experiment.

14 Reproducibility

The experiment used:

- Python `random` seed: 20

- NumPy random seed: 20
- PyTorch manual seed: 20
- Deterministic dataset subsampling (8,000 of 271,824 filtered pairs)
- Deterministic train/validation/test partitioning (6,400 / 800 / 800)
- Deterministic vocabulary construction from the sampled subset
- Seeded `DataLoader` batch shuffling
- Fixed model hyperparameters (Table 1)
- Fixed maximum sequence length (10 tokens)

The experiment was executed on **CPU**; no GPU/CUDA-specific determinism settings were required. Exact results can nevertheless vary across different hardware, library versions, and PyTorch releases despite the fixed seeds above.

15 Limitations

Several limitations should be considered.

First, the experiment uses a relatively simple whitespace tokenizer with accent-stripping rather than a modern subword tokenizer such as BPE or SentencePiece, and restricts sentences to non-letter characters plus `.`, `!`, and `?` – other punctuation (commas, apostrophes) is discarded during normalization.

Second, the models were trained on a subset of only 8,000 sentence pairs (of 271,824 available after length filtering) capped at 10 tokens per sentence, chosen to keep CPU training time tractable. This is the dominant reason absolute BLEU scores are low (under 11% for even the best model) compared with NMT systems trained on the full corpus or with larger models; the relative ranking between architectures is still informative, but the absolute scores should not be compared against literature values obtained on larger training sets.

Third, all four models use compact vanilla-RNN recurrent cells (128-dimensional hidden state) rather than gated cells (GRU/LSTM) or Transformer components; vanilla RNNs are more susceptible to vanishing gradients, which may partly explain the very low BLEU of Architectures 1–2 even on comparatively short (under-10-token) sequences.

Fourth, perplexity in this report is derived from training/validation cross-entropy loss under teacher forcing rather than from a teacher-forced pass over the held-out test set, so it is not perfectly comparable to the test-set BLEU and latency figures, which do use the test set.

Fifth, both non-attentional models (Architectures 1–2) and, to a lesser extent, the attention-based models begin overfitting well before epoch 40 (see Section 12), and no early stopping or additional regularization beyond $p = 0.1$ dropout was applied; final-epoch metrics may therefore understate what each architecture could achieve with early stopping on validation loss.

Sixth, the experiment uses a single random seed (20) and a single training run per architecture; no variance across seeds is reported.

Seventh, BLEU has well-known limitations and should not be interpreted as a complete measure of translation quality, particularly at the sentence lengths and data scale used here.

16 Final Result Summary

Table 4: *Best-performing architecture per evaluation criterion.*

Criterion	Best Architecture	Value
BLEU-4	Bahdanau Attention	10.67%
Validation Perplexity	Bahdanau Attention	35.10
Best Validation Perplexity	Bahdanau Attention	30.52
Mean Latency	Plain RNN	1.58 ms
Fewest Parameters	Plain RNN	1,803,324

The final interpretation should consider all evaluation dimensions rather than selecting an architecture solely from its BLEU score.

17 Discussion and Conclusion

This study compares four recurrent NMT models using the same 8,000-pair German–English dataset. The Plain RNN and Encoder–Decoder models performed very similarly, with BLEU-4 scores of 1.00% and 1.14%. This shows that simply separating the encoder and decoder did not make a big difference. Both models still have the fixed context-vector limitation. Adding attention gave a much bigger improvement. Bahdanau attention achieved 10.67% BLEU-4, while Luong attention achieved 3.72%. The overall ranking was:

Bahdanau Attention > Luong Attention > Encoder–Decoder > Plain RNN

The results show that attention was more important than simply separating the encoder and decoder. With attention, the decoder can focus on different parts of the source sentence while generating each word instead of depending only on one fixed context vector. Bahdanau attention performed better than Luong attention in this experiment.

These results should still be considered within the limits of this project. The models were trained using a single random seed, on only 8,000 sentence pairs, with sentences limited to 10 tokens, mainly to keep CPU training time manageable. Therefore, the BLEU scores are relatively low. However, the comparison still shows a clear trend: attention improved translation quality considerably, and Bahdanau attention gave the best results among the four models tested.

References

[1] Bahdanau, D., Cho, K., & Bengio, Y. (2015). *Neural Machine Translation by Jointly Learning to Align and Translate*. International Conference on Learning Representations (ICLR).

[2] Luong, M.-T., Pham, H., & Manning, C. D. (2015). *Effective Approaches to Attention-based Neural Machine Translation*. Proceedings of EMNLP.

[3] Sutskever, I., Vinyals, O., & Le, Q. V. (2014). *Sequence to Sequence Learning with Neural Networks*. Advances in Neural Information Processing Systems (NeurIPS).

[4] Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). *BLEU: A Method for Automatic Evaluation of Machine Translation*. Proceedings of ACL.

[5] ManyThings.org / Tatoeba Project. *German–English Sentence Pairs* (`deu.txt`). Retrieved from `https://www.manythings.org/anki/`